
REQUIREMENTS NOT MET

None.

PROBLEMS ENCOUNTERED

None Encountered.

PSEUDOCODE/FLOWCHARTS

SECTION 1

```
#define TABSIZE 16
#define OUT_TAB_START 0x3700
#include <avr/io.h>
int main(void){
// create an array of 8-bit value with a size of 16; replace ?? with the values in Table 1,
separated by commas
uint8_t IN_TAB [TABSIZE] = { data };
uint8_t *IN_TAB_P = &IN_TAB; // create a pointer to this table
// Note that using the // create a pointer to this table & above is not necessary since by
default, 8-bit arrays are
// pointers which point to the first element of the array.
// Thus, using both *IN_TAB_P = &IN_TAB; or *IN_TAB_P = IN_TAB; will produce the same
// functionality since IN_TAB is an address; however, to remain consistent we will
use the ampersand.

// Initialize a pointer to point to the output table at location 0x3700. Call it OUT_TAB_P
uint8_t OUT_TAB_P (pointer type)value OUT_TAB_START
```

SECTION 2

```
//To start the processing, we can use the following template:
while(1){
// Loop till null, i.e., loop while the values at that address are non-zero
// (null)
while(*IN_TAB_P){
// Check for the conditions
// Recall when using an *, we are analyzing the value at that
// pointer
// Dereference IN_TAB_P and check if bit 5 is set (See the Example
// 2).
// if bit 5 was set, then multiply by 2
if( value of IN_TAB_P & (1<<5) not 0 ){
value of IN_TAB_P = value of IN_TAB_P * 2
// check if the value at IN_TAB_P times two is greater than
// or equal to 70
if( value of IN_TAB_P >= 70 ) {
/*Store the result to address pointed by OUT_TAB_P */
// remember to alter the data at an address using a
// pointer, then you must use an (*). (See Example 1)
value of OUT_TAB_P = value of in IN_TAB_P

/*Increment OUT_TAB_P to move to next available space
*/

OUT_TAB_P += 1

}
}
else{
// if the input value (dereference IN_TAB_P) +3 is less than
//78
store the value to OUT_TAB_P
if( value of IN_TAB_P < 75 ){ // Same as IN_TAB_P+3 < 78
/*Store the result to address pointed by OUT_TAB_P */
value of OUT_TAB_P = (value of IN_TAB_P + 3)
```

```
                                /*Increment OUT_TAB_P to move to available space */
                                OUT_TAB_P += 1
                                }
                                }
    IN_TAB_P+=1; // increment IN_TAB_P, the next address in IN_TAB
    }

    return 0;
}
```

PROGRAM CODE

SECTION 1

```
/*
 * HW3.c
 *
 * Created: 7/11/2026 3:25:39 PM
 * Author : Evan Baessler
 */

#define TABSIZE 16
#define OUT_TAB_START 0x3700
#include <avr/io.h>
int main(void){
    // create an array of 8-bit value with a size of 16; replace ?? with the values in Table 1,
    // separated by commas
    uint8_t IN_TAB [TABSIZE] = { 201, 0x22, 051, 91, 0x91, 0x2A, 'U', 0130, '#', 0b01010101, 'J',
    0x21, 0x55, 32, 0b00011110, 0 };
    uint8_t *IN_TAB_P = &IN_TAB; // create a pointer to this table
    // Note that using the // create a pointer to this table & above is not necessary since by
    // default, 8-bit arrays are
    // pointers which point to the first element of the array.
    // Thus, using both *IN_TAB_P = &IN_TAB; or *IN_TAB_P = IN_TAB; will produce the same
    // functionality since IN_TAB is an address; however, to remain consistent we will use the
    // ampersand.

    // Initialize a pointer to point to the output table at location 0x3700. Call it OUT_TAB_P
    uint8_t *OUT_TAB_P = (uint8_t *)OUT_TAB_START; // Treats the pointer as a hex number
```

SECTION 2

```
//To start the processing, we can use the following template:
while(1){
    // Loop till null, i.e., loop while the values at that address are non-zero (not null)
    while(*IN_TAB_P){
        // Check for the conditions
        // Recall when using an *, we are analyzing the value at that pointer
        // Dereference IN_TAB_P and check if bit 5 is set (See the Example 2).
        //if bit 5 was set, then multiply the value by two!
        if( *IN_TAB_P & (1<<5) ){
            *IN_TAB_P = *IN_TAB_P * 2;
            // check if the value at IN_TAB_P times two is greater than or equal to 70
            if( *IN_TAB_P >= 70 ) {
                /*Store the result to address pointed by OUT_TAB_P */
                // remember to alter the data at an address using a pointer, then you
                must use an (*). (See Example 1)
                *OUT_TAB_P = *IN_TAB_P;

                /*Increment OUT_TAB_P to move to next available space */
                OUT_TAB_P += 1;
            }
        }
        else{
            // if the input value (dereference IN_TAB_P) +3 is less than 78, store the
            result in the output table.
            if( (*IN_TAB_P) < 75 ){
                /*Store the result to address pointed by OUT_TAB_P */
                *OUT_TAB_P = (*IN_TAB_P + 3);
                /*Increment OUT_TAB_P to move to available space */
```

```
        OUT_TAB_P += 1;
    }
}
IN_TAB_P++; // increment IN_TAB_P, the next address in IN_TAB
// should this instruction be somewhere else?
// Yes, we moved it up two rows INTO the loop.
}
return 0;
}
```

APPENDIX

Memory:	prog APP_SECTION	Address:	0x003700,data	
data 0x003700	52 54 46 4d 21 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	RTFM!		
data 0x003715	00 00			
data 0x00372A	00 00			
data 0x00373F	00 00			
data 0x003754	00 00			
data 0x003769	00 00			
data 0x00377E	00 00			
data 0x003793	00 00			
data 0x0037A8	00 00			

Figure 1: Output Table Filtered Values